

PBertKla + ML 스택킹 파이프라인

개선 전 / 개선 후 비교 보고서

Train·Test 메타 피처 분포 일관성 회복 및 정석 Stacked Generalization 적용

문서 정보	
대상 프로젝트	PBertKla v2 (단백질 라이신 락틸레이션 부위 예측)
대상 파이프라인	PBertKla 파인튜닝 + LightGBM/XGBoost/CatBoost 스택킹
변경 범위	Stage 2 (ML) — DL 메타 피처 처리 방식
DL 모델	ProteinBERT (epoch 92400, sample 23.5M, fine-tuned)
ML 모델	LightGBM + XGBoost + CatBoost (Soft-Voting Ensemble)
작성일	2026-04-24

1. 요약 (Executive Summary)

본 보고서는 PBertKla 파인튜닝 모델과 트리 기반 ML 모델 3종을 결합한 2단계 스택킹(stacked generalization) 파이프라인의 구조적 결함을 진단하고, 이를 정석적인 방식으로 개선한 내용을 정리한다.

발견된 문제: 기존 구조에서는 ML 학습 시점에 사용한 메타 피처(LightGBM OOF)와 ML 추론 시점에 사용한 메타 피처(transformer 단일 fold 예측)의 출처가 달라, ML 모델이 학습한 결합 규칙을 테스트에서 일반화하지 못한다. 그 결과 ML 출력이 DL 출력의 단순 변환에 가까워지고(상관계수 0.998), ML이 독립적으로 기여한다고 주장하기 어려운 상태였다.

적용한 개선: DL 5-fold 학습의 각 fold validation 예측을 모아 train OOF 메타 피처로 사용하고, test 메타 피처도 5-fold 평균 transformer 예측으로 통일하였다. 이로써 train과 test 모두 같은 transformer 분포의 메타 피처를 보게 되며, 방법론적으로 표준 stacked generalization (Wolpert, 1992)을 따른다.

2. 전체 파이프라인 구조

본 파이프라인은 두 단계로 분리된 sequential stacking 구조다. Stage 1에서 transformer 기반 DL 모델을 5-fold CV로 파인튜닝하고, Stage 2에서 그 출력을 메타 피처로 받아 트리 기반 ML 모델 3종을 학습한다. 최종 예측은 ML 3종의 soft voting ensemble로 산출한다.

2.1 단계별 역할

- Stage 1 (DL): 단백질 서열 자체로부터 contextual embedding을 학습하고, 라이신 락틸레이션 가능성에 대한 확률을 출력. 5-fold CV.
- Stage 2 (ML): DL 확률 + 수기로 추출한 서열 통계 피처(AAC, DPC, length)를 결합 입력으로 받아 LightGBM, XGBoost, CatBoost 각각을 Optuna 100 trials로 튜닝·학습. 마지막에 세 모델의 평균 확률로 ensemble.
- 두 단계의 연결 고리: Stage 1이 만든 DL 확률 벡터가 Stage 2의 입력 한 차원을 차지하는 메타 피처로 사용됨. 이 메타 피처를 어떻게 만들고 어떻게 정렬하는지가 본 보고서의 핵심 변경 지점.

3. Stage 1 — PBertKla 파인튜닝 (변경 없음)

DL 학습은 본 개선에서 변경되지 않았다. ProteinBERT pretrained weight (epoch 92400, sample 23.5M)을 backbone으로 사용하고, 각 데이터셋의 train CSV에 대해 5-fold StratifiedKfold (seed=42)를 수행한다. 각 fold는 동일한 공통 test CSV로 평가된다.

단계	설명
1. 전처리	train CSV → dropna() + drop_duplicates() (필수: ML과 정렬 기준)
2. Split	StratifiedKfold(n_splits=5, shuffle=True, random_state=42)
3. Stage A	Pretrained transformer freeze, 분류 head만 학습 (lr=1e-2)
4. Stage B	전체 unfreeze, fine-tune (lr=2e-3, val_loss 기반 early stopping)
5. Stage C	seq_len 1024로 늘려 1 epoch 추가 학습 (lr=1e-5)

단계	설명
6. 평가	각 fold가 공통 test set 예측 → fold_k/y_pred.npy 저장

본 단계의 산출물은 fold별 best_fine_tuning_model.h5 (가중치)와 y_pred.npy (공통 test set 예측)이다. 본 개선에서는 이 가중치를 그대로 재활용한다.

4. 핵심 변경 — ML 메타 피처 처리 방식

Stage 2에서 ML 모델이 보는 입력은 다음과 같이 구성된다.

피처 그룹	차원	출처	비고
AAC (Amino Acid Composition)	20	수기 통계	20개 표준 아미노산 각각의 빈도 비율
DPC (Dipeptide Composition)	400	수기 통계	20×20 = 400개 dipeptide 빈도 비율
Sequence length	1	수기 통계	정수 길이 (정규화 없음)
DL 메타 피처	1	PBertKla 출력	★ 본 보고서의 변경 대상
합계	422	—	—

421개의 sequence feature는 DL/ML 모두에서 동일하게 추출되어 변경 사항이 없다. 변경된 것은 마지막 422번째 차원, 즉 DL 메타 피처를 어떻게 만드는가이다.

4.1 변경 전 (Before) — Train·Test 메타 출처 불일치

기존 구조에서는 ML 학습용 메타 피처와 ML 추론용 메타 피처가 서로 다른 모델에서 유래했다. ML 학습 시점에는 transformer를 다시 돌릴 비용이 부담되어 sequence feature 기반 LightGBM의 OOF 예측으로 메타 피처를 대체했다. 그러나 ML 추론 시점에는 선택된 단일 fold의 transformer 예측을 그대로 사용했다.

시점	메타 피처 출처	분포 특성
ML 학습 (train)	Sequence feature 기반 LightGBM OOF	트리 모델의 sigmoid 출력 분포 (대체로 0.1~0.9 균등)
ML 추론 (test)	Transformer (PBertKla) 단일 fold 예측	transformer의 sigmoid 출력 분포 (양극단 0/1 쪽으로 더 치우침)
DL 5-fold 활용	5개 중 1개 fold만 inference에 사용	5-fold ensemble의 분산 감소 효과 폐기

구조적 결함: ML 모델은 학습 시점에 "422번째 피처가 LightGBM OOF의 분포"라는 전제로 분류 규칙을 학습하는데, 추론 시점에는 갑자기 "transformer의 분포"를 받게 된다. 분포 mismatch로 인해 ML이 학습한 결합 규칙(메타 피처와 sequence feature를 어떻게 섞을지)이 일반화되지 않으며, 모델은 보수적으로 메타 피처 값을 거의 그대로 신뢰하는 방향으로 수렴한다. 이로 인해 LightGBM/XGBoost 출력은 transformer 출력과 상관관계수 0.998 수준에 도달하여 "DL의 monotone 변환"에 가까운 동작을 보였다.

4.2 변경 후 (After) — 정식 Stacked Generalization

개선된 구조에서는 train과 test 모두 transformer 분포의 메타 피처를 보도록 통일한다. Train 메타는 DL 5-fold 학습에서 자연스럽게 발생하는 OOF (Out-Of-Fold) 예측을 모아서 사용하고, Test 메타는 5-fold transformer의 평균 예측을 사용한다. 이는 Wolpert(1992)가 제안한 stacked generalization의 정식 구현이다.

시점	메타 피처 출처	분포 특성
ML 학습 (train)	Transformer 5-fold OOF 예측	각 행이 "본인을 학습에 안 쓴 transformer fold"의 예측
ML 추론 (test)	Transformer 5-fold 평균 예측	5개 fold 예측의 평균 (분산 감소 효과 활용)
DL 5-fold 활용	5개 fold 모두 활용	OOF로 train, 평균으로 test → 두 단계 모두 정보 손실 없음

4.3 OOF (Out-Of-Fold) 예측이란?

5-fold CV에서, k번째 fold를 학습할 때 그 fold의 validation 부분(전체 train의 1/5)은 학습에 사용되지 않는다. 학습이 끝난 모델로 그 validation 부분을 예측한 결과를 OOF 예측이라 부른다. 5개 fold를 모두 돌면 train의 모든 행이 정확히 한 번씩 "본인을 학습에 사용하지 않은 모델"의 예측을 갖게 되며, 이 예측들을 train CSV의 원래 순서대로 모은 벡터가 OOF 벡터가 된다.

- 누수 없음: 각 행의 예측은 그 행을 학습에 쓰지 않은 모델에서 나온 → train 메타 피처가 부당하게 "답"을 알지 못함.
- 분포 일관성: 학습된 transformer의 sigmoid 출력 분포 → test 시점의 transformer 출력과 동일한 통계적 특성을 가짐.
- 구현 비용 0: 기존 5-fold checkpoint를 그대로 재활용하여 validation 부분만 inference하면 되므로, DL 재학습 없이 수십 분 내 생성 가능.

5. 변경 전 / 변경 후 종합 비교

비교 항목	변경 전 (Before)	변경 후 (After)
Train 메타 피쳐	LightGBM OOF (sequence features only)	Transformer 5-fold OOF
Test 메타 피쳐	Transformer 단일 fold 예측	Transformer 5-fold 평균 예측
Train·Test 분포 일치성	출처 다름 (LGBM vs Transformer)	동일 (Transformer · Transformer)
DL 5-fold 활용	1개 fold만 (4개 폐기)	5개 fold 모두
방법론 분류	비표준 (커스텀 변형)	표준 Stacked Generalization (Wolpert 1992)
ML의 학습 대상	LightGBM 출력 패턴 + sequence features	Transformer 출력 패턴 + sequence features
ML 결합 규칙의 일반화	Train·Test 분포 mismatch로 어려움	동일 분포 → 학습 규칙 그대로 활용 가능
DL ↔ ML 상관관계수 (LGBM/XGB) ≈ 0.998 (사실상 동일)		감소 예상 ($\approx 0.85 \sim 0.93$)
DL 자체를 다시 학습?	—	아니오. 기존 가중치 재활용
추가 산출물	fold_k/y_pred.npy (test 예측만)	+ fold_k/y_val_pred.npy, val_idx.npy, oof_pred.npy

6. 효과 및 함의

6.1 ML 모델의 역할 변화

변경 전 ML은 DL 출력에 강하게 anchored되어 사실상 DL 점수의 monotone calibrator로 기능했다. LightGBM/XGBoost의 출력은 DL 출력과 상관관계수 0.998로 거의 동일했고, tree 모델이 첫 split에서 메타 피쳐를 잡고 그 위에 미세 조정만 가하는 양상이었다. 변경 후에는 메타 피쳐가 transformer 분포로 일관되므로, ML이 학습한 "메타 피쳐와 sequence feature를 어떻게 결합할지"의 규칙이 테스트에서도 그대로 의미를 가진다. ML이 비로소 DL의 systematic bias를 sequence 통계로 보정하는 진짜 보조 모델로 기능할 여지가 생긴다.

6.2 방법론적 정합성

기존 구조는 stacking이라고 부르기에 train·test 메타 출처가 일치하지 않아 학술적 표준에서 벗어난 변형이었다. 변경 후 구조는 Wolpert(1992)의 stacked generalization 정의를 그대로 따르며, 논문이나 외부 검토에서 "왜 train OOF만 다른 모델로 만드는가?"라는 지적을 받지 않을 수 있다. 향후 다른 base learner를 추가하거나 stacker를 교체하는 확장도 표준 framework 위에서 가능하다.

6.3 5-fold ensemble 효과 회복

기존에는 DL 5-fold 학습의 산출물 중 1개 fold만 inference에 사용했으므로, 5-fold ensemble이 가져오는 분산 감소 효과를 폐기하고 있었다. 변경 후 test 메타도 5-fold 평균을 사용하므로, transformer 자체의 noise가 줄어든 baseline 위에 ML이 얹히는 구조가 된다. 이는 DL baseline의 개선과 ML 효과 회복이 동시에 발생하는 누적 이득을 낳는다.

6.4 한계 및 추가 개선 여지

- Sequence feature의 정보량 한계: AAC + DPC + length는 transformer가 이미 contextual embedding으로 흡수한 정보와 redundant. ML이 진짜 독립적 신호를 갖기 위해서는 ESM/ProtT5 embedding, conservation score, secondary structure 등 transformer와 직교한 피처를 도입하는 것이 다음 단계.
- Test 셋 크기: 외부 평가용 new_data.csv가 16개 샘플에 불과해 통계적 유의성 검증이 어려움. 구조 개선의 실제 이득을 측정하려면 더 큰 hold-out set이 필요.
- Ensemble 가중치: 현재 ensemble은 단순 평균. 검증셋에서 모델별 가중치를 학습하는 weighted soft voting으로 추가 개선 가능.

7. 본 개선의 산출물

산출물	위치	용도
DL OOF 생성 스크립트	Code/generate_oof.py	기존 5-fold 가중치를 재사용해 OOF 생성
Transformer OOF 벡터	results/data{1,2,3}/oof_pred.npy	ML 학습용 train 메타 피쳐
Fold별 validation 예측	results/data{1,2,3}/fold_k/y_val_pred.oof	OOF 조립을 위한 중간 산출물
ML 학습 코드 수정	Code/run_ML.py	OOF 로드 및 train·test 정렬 처리
ML 학습 실행 스크립트	kla_ML_train.sh	--dl_pred_dir 를 부모 디렉토리로 변경
변경 후 ML 결과	results/ML_output/data{1,2,3}_oof/	기존 결과(_oof 없는 폴더)와 분리 보존

8. 결론

본 개선은 PBertKla + ML 스택킹 파이프라인의 핵심 결함이었던 train·test 메타 피쳐 분포 불일치를 해결한다. DL을 재학습할 필요 없이 기존 5-fold 가중치를 재사용하여 OOF 예측을 생성하고, 이를 ML 학습의 train 메타 피쳐로 사용함으로써 stacking이 학술적 표준 정의를 만족하도록 만들었다. 이로써 ML 모델이 DL의 단순 복제기 역할에서 벗어나 진정한 보조 분류기로서 기여할 수 있는 구조적 토대가 마련되었으며, 향후 sequence feature 교체나 ensemble 가중치 학습 같은 추가 개선들이 의미 있게 평가될 수 있는 출발점을 확보하였다.